

GIT Commands

1. To set username and email ID for commits.

```
git config --global user.name "user1"
git config --global user.email user1@demo.lab
```

2. To set the default editor.

```
git config --global core.editor vim
```

3. By default Git will create a branch called master when you create a new repository with git init. From Git version 2.28 onwards, you can set a different name for the initial branch.

To set main as the default branch name do:

```
git config --global init.defaultBranch main
```

4. If you want to check your configuration settings, you can use the following command to list all the settings Git can find at that point

```
git config --list
```

5. To create a new Git repository inside a current directory, use following command.

```
git init
```

6. The main command you use to determine which files are in which state

```
git status
```

7. In order to begin tracking a new file, you use the command

```
git add filename
Or
git add -A
```

8. If you want to know exactly what you changed, not just which files were changed then you can use the following command.

```
git diff
```

9. To commit the changes use command

```
git commit -m "message for commit"
```

10. To remove a file from Git, you have to remove it from your tracked files (more accurately, remove it from your staging area) and then commit. The git rm command does that, and also removes the file from your working directory so you don't see it as an untracked file the next time around.

```
git rm filename
```

11. To keep the file in your working tree but remove it from your staging area.

```
git rm --cached filename
```

12. To display the entire commit history using the default format.

```
git log
```

13. Remove <file> from the staging area, but leave the working directory unchanged. This unstages a file without overwriting any changes.

```
git reset filename
```

14. List all of the branches in your repo.

```
git branch
```

15. To create a new branch.

```
git branch branch-name
```

16. To create a new branch and shift to that branch.

```
git checkout -b branch-name
```

17. To shift to an existing branch.

```
git checkout branch-name
```

18. To rename a branch.

```
git branch -M old-name new-name
```

19. To merge a branch with existing branch.

```
git merge branch-name
```

20. To delete a branch.

```
git -d branch-name
```

21. Create a new connection to a remote repo. After adding a remote, you can use <name> as a shortcut for <url> in other commands.

```
git remote add name url
```

Example

```
git remote add origin https://github.com/abc-user/repo1.git
```

22. To clone a remote repository.

```
git clone url
```

Example

```
git clone https://github.com/abc-user/repo1.git
```

23. Push the branch to <remote>, along with necessary commits and objects. Creates named branch in the remote repository if it doesn't exist.

```
git push remote branch-name
```

Example

```
git push origin main
```

24. Fetch the specified remote's copy of current branch and immediately merge it into the local copy.

```
git pull remote
```

Example

```
git pull origin main
```

25. To download commits, files, and references from a remote repository into the local repository. To retrieve the latest changes from the remote without automatically merging or integrating them into the working branch.

```
git fetch origin
```

Note :-

To exclude files from tracking by git, simply create a file by name .gitignore in the git repository directory (not inside .git). Add the filenames that you don't want to keep track of. However if you modify these files, you will not be able to revert to their earlier versions. The .gitignore file itself is tracked by the git.